

Doc 8.3 — Nullish Coalescing ?? & Optional Chaining ?.

Section 8 — Control Flow & Conditionals

1. Overview

JavaScript-e onek shomoy amader emon data niye kaj korte hoy jekhane kono value **thakteo pare, nao thakte pare**।

Example:

```
const user = {  
  name: "Shariar"  
};
```

Ekhane:

```
user.name
```

ache.

Kintu:

```
user.address
```

nei.

Abar API theke emon data aste pare:

```
const user = {  
  name: "Shariar",  
  address: null  
};
```

Ekhon directly:

```
user.address.city
```

likhle error hobe.

Ei type-er situation handle korar jonno JavaScript-e duita khub important operator ache:

```
??
```

and

```
?.
```

Main idea

Operator	Main purpose
<code>??</code>	<code>null</code> / <code>undefined</code> hole fallback value deya
<code>?.</code>	<code>null</code> / <code>undefined</code> hole safely property/method access kora

2. Nullish Value Ki?

`null` ebong `undefined` ke ekhane **nullish values** bola hoy.

```
const a = null;  
const b = undefined;
```

Duita-i nullish.

Kintu:

```
0  
""  
false  
NaN
```

egulo **nullish na**.

Eta `??` bujhar jonno extremely important.

3. Nullish Coalescing Operator `??`

Syntax:

```
leftValue ?? rightValue
```

Meaning:

Left side jodi `null` ba `undefined` hoy, tahole right side return koro!
Otherwise left side-i return koro!

Example:

```
const username = null ?? "Guest";  
  
console.log(username);
```

Output:

```
Guest
```

Because:

```
null
```

nullish.

Another example:

```
const username = "Shariar" ?? "Guest";  
  
console.log(username);
```

Output:

```
Shariar
```

Because `"Shariar"` nullish na.

4. `??` er Basic Mental Model

Eta mathay rakho:

```
left ?? right

      ↓

left === null
    OR
left === undefined

      ↓

right

otherwise
      ↓
left
```

Basically:

```
value ?? fallback
```

mane:

"Value ta jodi actually missing hoy, tokhon fallback use koro."

5. **null** Example

```
const age = null;

const result = age ?? 18;

console.log(result);
```

Output:

```
18
```

Because:

```
age === null
```

6. `undefined` Example

```
let age;

const result = age ?? 18;

console.log(result);
```

Output:

```
18
```

Because:

```
age === undefined
```

7. Non-nullish Values

Important:

```
const a = 0 ?? 100;
const b = "" ?? "Hello";
const c = false ?? true;
```

Output:

```
0
""
false
```

Why?

Because:

```
0      → not nullish
""     → not nullish
false  → not nullish
```

?? only checks:

```
null
undefined
```

8. ?? vs ||

Eta JavaScript-er one of the most important differences.

||

|| left side-er **falsy value** hole right side return korel
Falsy values-er moddhe ache:

```
false
0
-0
0n
""
null
undefined
NaN
```

But ?? only checks:

```
null
undefined
```

9. || vs ?? Example

```
const a = 0 || 100;
const b = 0 ?? 100;

console.log(a);
console.log(b);
```

Output:

```
100
0
```

Why?

```
||:
```

```
0 → falsy
```

So:

```
0 || 100
```

returns:

```
100
```

But:

```
0 ?? 100
```

checks whether `0` is:

```
null?
undefined?
```

No.

Therefore:

```
0
```

10. Empty String Example

```
const name1 = "" || "Guest";  
const name2 = "" ?? "Guest";  
  
console.log(name1);  
console.log(name2);
```

Output:

```
Guest  
""
```

Because empty string is falsy but **not nullish**.

11. Boolean Example

```
const isAdmin = false || true;  
const isAdmin2 = false ?? true;  
  
console.log(isAdmin);  
console.log(isAdmin2);
```

Output:

```
true  
false
```

`||`:

```
false → fallback
```

`??`:

```
false → valid value
```

12. NaN Example

```
const value1 = NaN || 50;
const value2 = NaN ?? 50;

console.log(value1);
console.log(value2);
```

Output:

```
50
NaN
```

Because:

```
NaN → falsy
NaN → not nullish
```

13. Comparison Table

Value	Falsy?	Nullish?	<code>value "X"</code>	<code>value ?? "X"</code>
false	✓	✗	"X"	false
0	✓	✗	"X"	0
""	✓	✗	"X"	""
NaN	✓	✗	"X"	NaN
null	✓	✓	"X"	"X"
undefined	✓	✓	"X"	"X"
"hello"	✗	✗	"hello"	"hello"

Golden rule

```
|| → falsy check
?? → null/undefined check
```

14. Real-World Example — User Age

Dhoro API theke:

```
const user = {  
  age: 0  
};
```

Age unusual holeo technically valid numeric value hote pare.

Wrong:

```
const age = user.age || 18;  
  
console.log(age);
```

Output:

```
18
```

But maybe amader actual value preserve kora dorkar.

Better:

```
const age = user.age ?? 18;  
  
console.log(age);
```

Output:

```
0
```

15. Real-World Example — Product Price

```
const product = {  
  discount: 0  
};
```

Wrong:

```
const discount = product.discount || 10;
```

Result:

```
10
```

But actual discount:

```
0
```

So:

```
const discount = product.discount ?? 10;
```

Result:

```
0
```

16. When Should You Use `||`?

Jodi tomar requirement hoy:

"Jekono falsy value hole fallback use korbo."

Then:

```
const value = input || "Default";
```

Useful.

Example:

```
const search = userInput || "all";
```

Empty string hole `"all"` use korte chaile `||` appropriate.

17. When Should You Use `???`?

Jodi requirement hoy:

"Only value completely missing hole fallback use korbo."

Then:

```
const value = input ?? "Default";
```

Especially useful for:

- API data
- configuration
- database values
- optional properties
- numeric values
- boolean settings

18. `??` Short-Circuiting

`??` ekta short-circuit operator.

Example:

```
const value = "Hello" ?? expensiveFunction();
```

`"Hello"` nullish na, tai:

```
expensiveFunction()
```

call hobe na.

Example:

```
const value = null ?? expensiveFunction();
```

Ekhane `expensiveFunction()` call hobe.

Mental model

```
left valid?  
  ↓  
  YES → left return  
  NO  → right evaluate
```

19. Nullish Assignment `??=`

Closely related operator:

```
??=
```

Meaning:

Variable/property nullish hole new value assign koro!

Example:

```
let username;  
  
username ??= "Guest";  
  
console.log(username);
```

Output:

```
Guest
```

Because:

```
username === undefined
```

But:

```
let username = "Shariar";

username ??= "Guest";

console.log(username);
```

Output:

```
Shariar
```

Existing non-nullish value overwrite hoyni.

20. `??=` vs `||=`

```
let count = 0;

count ||= 10;

console.log(count);
```

Output:

```
10
```

Because `0` falsy.

But:

```
let count = 0;

count ??= 10;

console.log(count);
```

Output:

```
0
```

Because `0` nullish na.

21. Object-er Sathe ??=

```
const settings = {  
  theme: "dark"  
};  
  
settings.theme ??= "light";  
settings.language ??= "en";  
  
console.log(settings);
```

Result:

```
{  
  theme: "dark",  
  language: "en"  
}
```

theme already available, tai change hoyni. language missing, tai assign hoyeche.

22. ?? and Operator Precedence

?? er precedence niye careful thakte hobe.

Example:

```
const result = null ?? "Hello" ? "Yes" : "No";
```

Confusing hote pare.

Readability-r jonno parentheses use koro:

```
const result = (null ?? "Hello") ? "Yes" : "No";
```

Best practice:

Complex expression hole parentheses use koro.

23. `??` with `||` or `&&`

JavaScript-e directly mix kora allowed na:

```
a ?? b || c;
```

Eta syntax error.

Similarly:

```
a || b ?? c;
```

or:

```
a ?? b && c;
```

invalid.

Correct way

Parentheses use koro:

```
(a ?? b) || c;
```

or:

```
a ?? (b || c);
```

Similarly:

```
(a ?? b) && c;
```

Golden rule

`??` er sathe `||` / `&&` mix korle intention explicitly parentheses diye bolo.

24. Optional Chaining `?.`

Ebar ashi second major operator-e:

```
?.
```

Optional chaining-er purpose:

Object/property/function nullish hole error na diye safely `undefined` return kora.

MDN-er definition onujayi `?.` nullish (`null` / `undefined`) reference pele chain short-circuit kore `undefined` return kore.

25. Normal Property Access Problem

```
const user = null;

console.log(user.name);
```

Error:

```
TypeError
```

Because `user` null.

26. Optional Chaining Solution

```
const user = null;

console.log(user?.name);
```

Output:

```
undefined
```

No TypeError.

27. Basic Syntax

```
object?.property
```

Example:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(user?.name);
```

Output:

```
Shariar
```

If property doesn't exist:

```
console.log(user?.email);
```

Output:

```
undefined
```

28. Nested Optional Chaining

Suppose:

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka"  
  }  
};
```

You can write:

```
console.log(user?.address?.city);
```

Output:

```
Dhaka
```

Now:

```
const user = {  
  name: "Shariar"  
};
```

Then:

```
console.log(user?.address?.city);
```

Output:

```
undefined
```

No error.

29. Why Multiple `?.` ?

Eta important.

```
user?.address?.city
```

First:

```
user?.address
```

Then:

```
?.city
```

Both levels missing hote pare.

For example:

```
user = null
```

or:

```
user.address = null
```

or:

```
user.address.city = undefined
```

Optional chaining safely handle korte pare.

30. Without Optional Chaining

Old-style code:

```
if (
  user &&
  user.address &&
  user.address.city
) {
  console.log(user.address.city);
}
```

Optional chaining:

```
console.log(user?.address?.city);
```

Much cleaner.

31. Array Optional Access

Optional chaining array-er satheo use kora jay.

Syntax:

```
array?.[index]
```

Example:

```
const users = ["Shariar", "Duke"];

console.log(users?.[0]);
```

Output:

```
Shariar
```

If array itself missing:

```
let users;

console.log(users?.[0]);
```

Output:

```
undefined
```

No error.

32. Dynamic Property Access

Bracket notation-er satheo use kora jay:

```
const key = "name";

const user = {
  name: "Shariar"
};

console.log(user?.[key]);
```

Output:

Shariar

Syntax:

```
object?.[expression]
```

33. Optional Function Call

Dhoro ekta callback thakteo pare, nao thakte pare.

```
function handleSuccess() {  
  console.log("Success");  
}
```

You can do:

```
handleSuccess?.();
```

Function thakle call hobe.

Function na thakle:

```
undefined
```

return korbe instead of throwing because of the missing function.

34. React Callback Example

Very common pattern:

```
function Button({ onClick }) {  
  const handleClick = () => {  
    onClick?.();  
  };  
  
  return <button onClick={handleClick}>Click</button>;  
}
```

`onClick` pass korle:

```
onClick()
```

call hobe.

Na korle:

```
onClick?.()
```

safe thakbe.

35. Optional Method Call

```
const user = {  
  greet() {  
    return "Hello";  
  }  
};  
  
console.log(user.greet?.());
```

Output:

```
Hello
```

Method missing hole:

```
const user = {};  
  
console.log(user.greet?.());
```

Output:

```
undefined
```

36. Important Difference

Ei duita same na:

```
user?.greet();
```

and:

```
user.greet?.();
```

First

```
user?.greet();
```

Checks:

```
user
```

nullish kina.

Second

```
user.greet?.();
```

Checks:

```
greet
```

function exists kina.

Jodi duita-i uncertain hoy:

```
user?.greet?.();
```

Eta safest.

37. Optional Chaining Does NOT Mean Everything Is Safe

Important misconception:

```
user?.name.toUpperCase();
```

It automatically puts a chain safe guard in every case.

Better:

```
user?.name?.toUpperCase();
```

Because `user` exist or not:

```
user.name
```

`null` or `undefined` is possible.

Example:

```
const user = {
  name: null
};

console.log(user?.name?.toUpperCase());
```

Output:

```
undefined
```

38. Continuous Chain Concept

Example expression:

```
user?.address?.city
```

continuous optional chain.

But:

```
(user?.address).city
```

different.

Jodi:

```
user = null;
```

Then:

```
(user?.address).city
```

error dite pare.

Because `user?.address` first `undefined` return korbe, tarpor:

```
undefined.city
```

access korar chesta hobe.

MDN-o ei grouped-chain behavior-er warning dey.

Better:

```
user?.address?.city
```

39. Optional Chaining with `??`

Ei combination ta **extremely important**:

```
?. + ??
```

Example:

```
const user = null;

const city = user?.address?.city ?? "Unknown";

console.log(city);
```

Output:

Unknown

40. Mental Model of `?. ??`

```
user?.address?.city ?? "Unknown"
```

Basically:

```
Can I safely reach city?
  ↓
  yes
  ↓
Is city nullish?
 /      \
yes      no
  ↓      ↓
"Unknown"  city
```

Eta API data handling-e huge useful.

41. API Response Example

Suppose API response:

```
const response = {
  data: {
    user: {
      profile: {
        name: "Shariar"
      }
    }
  }
};
```

You can write:

```
const name = response?.data?.user?.profile?.name ?? "Unknown";

console.log(name);
```

Output:

```
Shariar
```

If response incomplete:

```
const response = {
  data: {}
};
```

Then:

```
const name = response?.data?.user?.profile?.name ?? "Unknown";
```

Output:

```
Unknown
```

No crash.

42. React API Example

Suppose:

```
const user = data?.user;
```

Then JSX:

```
{data?.user?.name ?? "Guest"}
```

This means:

user/name thakle show koro, otherwise "Guest" show koro.

43. Product Example

```
const product = {  
  title: "Laptop",  
  pricing: {  
    amount: 85000  
  }  
};
```

Use:

```
const price = product?.pricing?.amount ?? 0;  
  
console.log(price);
```

Output:

```
85000
```

If pricing missing:

```
const product = {  
  title: "Laptop"  
};
```

Then:

```
const price = product?.pricing?.amount ?? 0;
```

Output:

```
0
```

44. User Profile Example

```
const user = {  
  profile: {  
    name: "Shariar"  
  }  
};  
  
const name = user?.profile?.name ?? "Guest";  
const city = user?.profile?.address?.city ?? "Unknown";
```

Result:

```
name → "Shariar"  
city → "Unknown"
```

45. ?. vs &&

Before optional chaining, developers often wrote:

```
const city =  
  user &&  
    user.address &&  
      user.address.city;
```

Now:

```
const city = user?.address?.city;
```

Much cleaner.

But subtle difference aache.

`&&` falsy value-er upor short-circuit kore. `?.` specifically nullish value (`null` / `undefined`) check kore.

46. Example: `&&` vs `?.`

```
const obj = {  
  value: 0  
};
```

With:

```
const result = obj.value && obj.value.toString();
```

Because `0` falsy:

```
0
```

return korte pare.

Optional chaining:

```
const result = obj.value?.toString();
```

`0` nullish na, so method call korte parbe:

```
"0"
```

47. `?.` + `??` vs `||`

Suppose:

```
const user = {  
  age: 0  
};
```

Using:

```
const age = user?.age || 18;
```

Result:

```
18
```

But:

```
const age = user?.age ?? 18;
```

Result:

```
0
```

Again:

```
|| → falsy  
?? → nullish
```

48. Common API Pattern

Modern JavaScript/React-e frequently dekhba:

```
const value = response?.data?.result?.value ?? defaultValue;
```

Eta ekshathe duita problem solve kore:

?.

Missing nested property-er jonno crash prevent kore.

??

Missing result hole default value dey.

49. Optional Chaining with Array Data

```
const response = {
  users: [
    {
      name: "Shariar"
    }
  ]
};
```

You can write:

```
const firstUser = response?.users?.[0]?.name ?? "Unknown";
```

Output:

```
Shariar
```

If users missing:

```
const response = {};
```

Then:

```
Unknown
```

50. Optional Callback Pattern

```
function processData(data, onSuccess, onError) {  
  try {  
    console.log(data);  
  
    onSuccess?.(data);  
  } catch (error) {  
    onError?.(error);  
  }  
}
```

Callback provided hole call hobe.

Na hole error hobe na.

51. Optional Chaining Does Not Fix Undeclared Variables

Very important.

Eta:

```
someUndefinedVariable?.name;
```

safe na jodi variable-ta actually declare-i kora na hoy.

It can throw:

```
ReferenceError
```

Optional chaining declared variable-er value `null` / `undefined` hole useful.

MDN explicitly notes that optional chaining cannot be used to rescue a non-declared root identifier.

52. Optional Chaining Does Not Hide Every Error

Example:

```
const user = {
  greet: "hello"
};

user.greet?.();
```

`greet` exists, but function na.

Eta automatically every TypeError prevent korbe na.

`?.` primarily checks whether the value is nullish before calling.

So:

```
missing function → safe
non-function value → TypeError
```

53. Optional Chaining Assignment Invalid

Eta kora jay na:

```
user?.name = "Shariar";
```

Eta invalid.

Optional chaining reading/accessing-er jonno, direct assignment target hisebe use kora jay na.

Correct:

```
if (user) {
  user.name = "Shariar";
}
```

54. ?? vs Default Parameter

Closely related concept.

Function:

```
function greet(name = "Guest") {  
  console.log(name);  
}
```

Default parameter use hoy mainly jekhoni argument:

```
undefined
```

hoy.

Example:

```
greet();
```

→ "Guest"

But:

```
greet(null);
```

→ null

So default parameter `null` handle kore na.

??:

```
const finalName = name ?? "Guest";
```

handles both:

```
null  
undefined
```

55. ?? vs Destructuring Default

Example:

```
const { name = "Guest" } = user;
```

Again default mainly `undefined` hole apply hoy.

If:

```
const user = {  
  name: null  
};
```

Then:

```
const { name = "Guest" } = user;  
  
console.log(name);
```

Output:

```
null
```

But:

```
const name = user.name ?? "Guest";
```

Output:

```
Guest
```

Eta distinction interview-e important.

56. Real-World Settings Example

```
const settings = {  
  notifications: false,  
  volume: 0  
};
```

Wrong:

```
const notifications = settings.notifications || true;  
const volume = settings.volume || 50;
```

Result:

```
notifications → true  
volume → 50
```

But actual values lost hoye gelo.

Better:

```
const notifications = settings.notifications ?? true;  
const volume = settings.volume ?? 50;
```

Result:

```
notifications → false  
volume → 0
```

57. Configuration Example

```
const config = {  
  port: 0  
};  
  
const port = config.port ?? 3000;  
  
console.log(port);
```

Output:

0

0 ke valid value hisebe preserve korbe.

58. Nested API Data — Full Example

```
const apiResponse = {
  data: {
    user: {
      profile: {
        name: "Shariar",
        age: 31
      }
    }
  }
};

const name =
  apiResponse?.data?.user?.profile?.name ?? "Unknown";

const age =
  apiResponse?.data?.user?.profile?.age ?? 0;

console.log(name);
console.log(age);
```

Output:

Shariar
31

59. Missing API Data

```
const apiResponse = {
  data: null
};

const name =
  apiResponse?.data?.user?.profile?.name ?? "Unknown";
```

Output:

```
Unknown
```

No crash.

60. React JSX Example

```
function UserCard({ user }) {
  return (
    <div>
      <h2>{user?.name ?? "Guest"}</h2>

      <p>
        {user?.profile?.city ?? "City not available"}
      </p>
    </div>
  );
}
```

Very common real-world pattern.

61. Next.js/API Example

API response-er structure sometimes uncertain:

```
const username =  
  response?.data?.user?.username ?? "Anonymous";
```

Instead of:

```
let username = "Anonymous";  
  
if (  
  response &&  
  response.data &&  
  response.data.user &&  
  response.data.user.username  
) {  
  username = response.data.user.username;  
}
```

Modern syntax much cleaner.

62. Optional Chaining + Function Return

```
function getUser() {  
  return null;  
}  
  
const name = getUser()?.profile?.name ?? "Guest";  
  
console.log(name);
```

Output:

```
Guest
```

63. Method Chaining

```
const user = {  
  name: "shariar"  
};  
  
const result =  
  user?.name?.toUpperCase?.() ?? "UNKNOWN";  
  
console.log(result);
```

Output:

```
SHARIAR
```

64. Common Mistake — Using `||` Everywhere

✗:

```
const quantity = product.quantity || 1;
```

If:

```
quantity = 0
```

then result becomes:

```
1
```

Maybe wrong.

Better when `0` is valid:

```
const quantity = product.quantity ?? 1;
```

65. Common Mistake — Forgetting `?.` at Deeper Level

✗:

```
user?.address.city
```

If:

```
user = {  
  address: null  
};
```

then:

```
TypeError
```

Better:

```
user?.address?.city
```

66. Common Mistake — Thinking `?.` Checks Truthiness

It doesn't.

```
const obj = {  
  value: 0  
};  
  
console.log(obj?.value);
```

Output:

```
0
```

Because:

```
0 ≠ null  
0 ≠ undefined
```

67. Common Mistake — Overusing Optional Chaining

Don't blindly write:

```
user?.profile?.name?.first?.something?.value
```

everywhere.

First understand whether those properties are actually optional.

Optional chaining is useful when absence is genuinely possible.

68. Common Mistake — Overusing ??

Don't use:

```
const name = name ?? "Guest";
```

automatically everywhere.

If empty string should also be considered invalid:

```
const name = name || "Guest";
```

might be more appropriate.

Choice depends on the business rule.

69. || vs ?? — Business Logic

Think like this:

Question 1

"Value ta ki missing?"

Use:

??

Question 2

"Value ta ki falsy?"

Use:

||

70. ?. vs &&

Question

"Object/property nullish hole crash prevent korte chai?"

Use:

?.

instead of verbose:

&&

71. Complete Mental Model

Three operators:

```
?.    → safely access
??    → fallback for null/undefined
||    → fallback for falsy
```

Example:

```
const city =  
  user?.profile?.address?.city ?? "Unknown";
```

Breakdown:

```
?. → safely navigate  
?? → missing value hole fallback
```

72. One-Line Master Pattern

Ei pattern ta strongly memorize korte paro:

```
const value = object?.deep?.property ?? defaultValue;
```

Example:

```
const username =  
  response?.data?.user?.name ?? "Guest";
```

This pattern real-world frontend development-e extremely common.

73. Practice — Output Prediction

Question 1

```
console.log(null ?? "Hello");
```

What is the output?

Question 2

```
console.log(0 ?? 100);
```

Question 3

```
console.log(0 || 100);
```

Question 4

```
console.log("" ?? "Guest");
```

Question 5

```
console.log("" || "Guest");
```

Question 6

```
const user = null;

console.log(user?.name);
```

Question 7

```
const user = {
  profile: {
    name: "Shariar"
  }
};

console.log(user?.profile?.name);
```

Question 8

```
const user = {};

console.log(user?.profile?.name ?? "Guest");
```

Question 9

```
const user = {  
  age: 0  
};  
  
console.log(user?.age || 18);  
console.log(user?.age ?? 18);
```

Question 10

```
const user = {  
  name: null  
};  
  
const name = user?.name ?? "Guest";  
  
console.log(name);
```

74. Practice — Find the Bug

Problem 1

```
const user = {  
  profile: null  
};  
  
console.log(user?.profile.name);
```

What's wrong?

Problem 2

```
const count = 0;

const result = count || 10;

console.log(result);
```

If `0` is a valid value, how would you fix it?

Problem 3

```
const data = null;

const result = data.value ?? "Unknown";
```

How can you make it safe?

Problem 4

```
const user = {
  callback: "hello"
};

user.callback?.();
```

Is this guaranteed to be safe? Why/why not?

75. Coding Challenge 1 — Safe User Data

Create:

```
const user = {
  profile: {
    name: "Shariar"
  }
};
```

Then safely extract:

```
name
  email
  city
```

Default values:

```
name → "Guest"
  email → "No email"
  city → "Unknown city"
```

Expected:

```
{
  name: "Shariar",
  email: "No email",
  city: "Unknown city"
}
```

76. Coding Challenge 2 — Product Data

Given:

```
const product = {
  title: "Laptop",
  pricing: {
    price: 85000
  }
};
```

Safely extract:

```
title
price
discount
currency
```

Defaults:

```
title → "Unknown product"
price → 0
discount → 0
currency → "BDT"
```

Use:

```
?.
```

and:

```
??
```

77. Coding Challenge 3 — API Response

Given:

```
const response = {
  data: {
    user: {
      profile: {
        name: "Shariar",
        age: 31
      }
    }
  }
};
```

Create:

```
const userInfo = {
  name: ...,
  age: ...,
  city: ...
};
```

Expected:

```
{  
  name: "Shariar",  
  age: 31,  
  city: "Unknown"  
}
```

78. Coding Challenge 4 — Settings

Given:

```
const settings = {  
  volume: 0,  
  notifications: false  
};
```

Use `??` to provide defaults:

```
volume → 50  
notifications → true  
theme → "light"
```

Expected:

```
{  
  volume: 0,  
  notifications: false,  
  theme: "light"  
}
```

79. Interview Questions

Beginner

1. What is the nullish coalescing operator?
2. What does `??` check?
3. What is the difference between `??` and `||`?
4. What are nullish values?

5. What does optional chaining do?
 6. What does `user?.name` return if `user` is `null`?
 7. Can `?.` be used with arrays?
 8. What does `func?.()` do?
-

Intermediate

9. Why does `0 ?? 10` return `0`?
 10. Why does `0 || 10` return `10`?
 11. Why is `user?.profile?.name` useful?
 12. How do `?.` and `??` work together?
 13. What is `??=`?
 14. What happens if you directly mix `??` with `||` without parentheses?
 15. What's the difference between `user?.method()` and `user.method?.()`?
 16. Does optional chaining protect an undeclared variable?
 17. Does optional chaining make every operation after it safe?
-

Advanced

18. Explain the difference between:

```
user?.name
```

and:

```
user && user.name
```

19. Explain:

```
user?.profile?.age ?? 18
```

step by step.

20. Why is this invalid?

```
a ?? b || c
```

21. What's the difference between:

```
const value = x ?? 10;
```

and:

```
const value = x || 10;
```

22. Explain the difference between `??=` and `||=`.

80. Quick Revision Table

Concept	Meaning
<code>??</code>	fallback only for <code>null</code> / <code>undefined</code>
<code> </code>	fallback for any falsy value
<code>?.</code>	safely access property/method
<code>obj?.prop</code>	safe property access
<code>obj?.[key]</code>	safe dynamic property access
<code>arr?.[0]</code>	safe array access
<code>fn?.()</code>	safe function call
<code>obj.method?.()</code>	safe method call
<code>??=</code>	assign only if nullish
<code>?.</code> + <code>??</code>	safe access + fallback

81. Most Important Examples

`??`

```
const name = null ?? "Guest";
```

Result:

Guest



```
const name = "" || "Guest";
```

Result:

Guest



```
const city = user?.address?.city;
```



```
const city = user?.address?.city ?? "Unknown";
```



```
let name;  
  
name ??= "Guest";
```

82. Golden Rules

Rule 1

?? শূন্য null এবং undefined check kore.

Rule 2

|| সব falsy value-er jonno fallback dite pare.

Rule 3

`0`, `""`, `false`, `NaN` nullish na.

Rule 4

Nested object safely access korte `?.` use koro.

Rule 5

`?.` + `??` is one of the most useful combinations for API/frontend code.

```
const value = data?.user?.profile?.name ?? "Guest";
```

Rule 6

`??` er sathe `||` / `&&` directly mix korle parentheses use koro.

Rule 7

`?.` undeclared variable-er `ReferenceError` prevent kore na.

Rule 8

`?.` assignment-er left-hand side hisebe use kora jay na.

83. Final Mental Model

Ei 3 ta question korlei operator choose korte parbe:

1. Value missing?

↓

`??`

2. Nested property safely access korte hobe?

↓

`?.`

3. Any falsy value hole fallback chai?

↓
||

Example

```
const username =  
  response?.data?.user?.name ?? "Guest";
```

Read it as:

"Response-er data-er user-er name safely access koro. Name `null` / `undefined` hole `Guest` use koro."

84. Section 8 — Complete Revision

Ekhon Section 8-er 3 ta docs:

SECTION 8 – CONTROL FLOW & CONDITIONALS

- └─  Doc 8.1
 - | if / else if / else
- └─  Doc 8.2
 - | switch / case / default
 - | Ternary Operator
- └─  Doc 8.3
 - | Nullish Coalescing ??
 - | Optional Chaining ?.

Overall mental model:

`if / else`

↓

Complex condition-based decisions

`switch`

↓

Multiple known cases

`ternary`

↓

Short inline condition

`??`

↓

Fallback for null / undefined

`?.`

↓

Safe access through nullable data

85. Section 8 Mastery Checklist

Before moving to the next JavaScript section, make sure you can explain these without notes:

- ☐ `if`
- ☐ `else if`
- ☐ `else`
- ☐ Nested conditions
- ☐ Guard clauses
- ☐ `switch`
- ☐ `case`
- ☐ `break`
- ☐ `default`
- ☐ Fall-through
- ☐ Ternary operator
- ☐ Nested ternary
- ☐ `??`
- ☐ `??` vs `||`
- ☐ Nullish values

- ☐ `??=`
- ☐ `?.`
- ☐ Nested optional chaining
- ☐ Optional array access
- ☐ Optional function call
- ☐ Optional method call
- ☐ `?.` + `??`
- ☐ API response handling
- ☐ React conditional/data rendering
- ☐ `??` with `&&` / `||`
- ☐ Common optional chaining mistakes



Final Takeaway

JavaScript-er modern code-e ei pattern-gulo khub frequently dekhba:

```
const name = user?.name ?? "Guest";
```

```
const city = user?.profile?.address?.city ?? "Unknown";
```

```
const firstUser = response?.data?.users?.[0]?.name ?? "No user";
```

```
callback?.();
```

Egulo properly bujhe gele API response, React component, form data, configuration, database response—sob jaygay **missing/nullish data safely handle** korte parba.

One-line memory trick:

```
?.. → safely go inside
??  → fallback if missing
||  → fallback if falsy
```

Ei tin ta clear thakle modern JavaScript-er conditional/data-access code onek easier hoye jabe.